



UNIVERSITEIT VAN AMSTERDAM



Bachelor Thesis Physics and Astronomy

---

## Searching for ultra-fast radio bursts using TransientX

---

*By:*

**Joël van Ingen**

*Conducted between April 2, 2024 & July 12, 2024*

|                            |                         |
|----------------------------|-------------------------|
| <i>Student number:</i>     | 12846309                |
| <i>Research institute:</i> | API                     |
| <i>Research group:</i>     | AstroFlash              |
| <i>Supervisor:</i>         | Prof. dr. Jason Hessels |
| <i>Daily supervisor:</i>   | Dr. Kaustubh Rajwade    |
| <i>Examiner:</i>           | Prof. dr. Ralph Wijers  |

### **Abstract**

Fast Radio Bursts (FRBs) are brief, intense radio pulses originating from distant galaxies. They are characterised by their high Dispersion Measure (DM), and provide valuable insights into extreme astronomical phenomena and interstellar and intergalactic matter. Despite their unknown origin, many theories suggest a connection to neutron stars. Detecting FRBs is challenging due to the abundance of Radio Frequency Interference (RFI), which can cause false positives. FRBs have been detected on a range of timescales, going down to mere microseconds. Detecting FRBs on that timescale requires sensitive telescopes and efficient data processing algorithms. TransientX, a new CPU-based software tool, offers a promising pipeline for FRB detection. This thesis benchmarks TransientX's performance on high-resolution datasets, aiming for efficient data analysis.

### **Abstract**

Fast Radio Bursts (FRBs) zijn korte radioflitsen die we op telescopen waarnemen. Deze flitsen komen van buiten ons eigen sterrenstelsel en kunnen van enkele seconden tot enkele microseconden duren - een miljoen keer korter dan een seconde. Het bestuderen van deze flitsen leert ons veel over gebeurtenissen in het universum in verre sterrenstelsels, en over ruimtestof dat zich tussen sterren en sterrenstelsels bevindt. Het detecteren van FRBs is moeilijk; ze komen zelden voor en we moeten veel telescoopdata doorzoeken voor we ze vinden. Daarnaast hebben telescopen veel last van interferentie die van de Aarde afkomstig is, wat detectie-algoritmes kan verwarren. Bijna elke flits die gedetecteerd wordt, is afkomstig van deze interferentie. We hebben dus goede algoritmes en efficiënte programma's nodig om de data te kunnen doorzoeken. Recent is een programma genaamd TransientX uitgekomen, en is gemaakt om dit soort astronomische data efficiënt te doorzoeken. In deze scriptie testen we dit programma om te beoordelen hoe goed het werkt.

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b>  |
| 1.1      | TransientX . . . . .                               | 4         |
| 1.2      | Motivation . . . . .                               | 4         |
| <b>2</b> | <b>Observations</b>                                | <b>5</b>  |
| 2.1      | Environment . . . . .                              | 5         |
| 2.2      | Datasets . . . . .                                 | 6         |
| 2.2.1    | Trial datasets . . . . .                           | 6         |
| 2.2.2    | High time resolution dataset . . . . .             | 6         |
| <b>3</b> | <b>Analysis</b>                                    | <b>6</b>  |
| 3.1      | Trial Runs . . . . .                               | 6         |
| 3.2      | High-time-resolution run . . . . .                 | 7         |
| 3.3      | TransientX Benchmarks . . . . .                    | 7         |
| <b>4</b> | <b>Results</b>                                     | <b>8</b>  |
| 4.1      | Trial datasets . . . . .                           | 8         |
| 4.1.1    | Dataset 1 . . . . .                                | 8         |
| 4.1.2    | Dataset 2 . . . . .                                | 8         |
| 4.2      | High-time-resolution dataset . . . . .             | 8         |
| 4.3      | TransientX Benchmarks . . . . .                    | 9         |
| 4.3.1    | Time-downsampling benchmarks . . . . .             | 9         |
| 4.3.2    | S/N threshold benchmarks . . . . .                 | 10        |
| 4.3.3    | DM step count benchmarks . . . . .                 | 10        |
| <b>5</b> | <b>Discussion</b>                                  | <b>13</b> |
| 5.1      | Trial runs and high-time-resolution run . . . . .  | 13        |
| 5.2      | Benchmark results . . . . .                        | 13        |
| 5.3      | TransientX stability . . . . .                     | 14        |
| 5.4      | Bottlenecks . . . . .                              | 15        |
| 5.4.1    | Causes of bottlenecks . . . . .                    | 16        |
| 5.4.2    | Mitigation of bottlenecks . . . . .                | 17        |
| 5.4.3    | Potential TransientX improvements . . . . .        | 18        |
| 5.4.4    | Search strategies . . . . .                        | 18        |
| 5.5      | Effective real-world usage of TransientX . . . . . | 20        |
| 5.6      | Future research . . . . .                          | 20        |
| <b>6</b> | <b>Conclusions</b>                                 | <b>21</b> |
| <b>7</b> | <b>Acknowledgements</b>                            | <b>22</b> |

# 1 Introduction

Fast radio bursts (FRBs) are brief radio flashes that have been observed to originate from distant galaxies. Since their discovery in 2007, they have captured the attention of astronomers and physicists due to their potential to provide insights into extreme astrophysical phenomena. The first FRB, detected during a search for pulsars in the Magellanic Clouds, was identified as a single burst of radio emission lasting less than 5 milliseconds [10], and originated from far past the Magellanic Clouds. Most FRBs seem to be one-off events, but a small fraction of FRBs are known to repeat [6]. FRBs have, so far, been found at frequency ranges from 110 MHz [17] up to 9 GHz [8]

FRBs are characterised by their high dispersion measure (DM), which is an observed sweep in the incoming signal where the higher radio frequencies arrive earlier than the lower frequencies. This delay is caused by free electrons in the interstellar and intergalactic medium, which causes the speed of light travelling through this medium to be frequency-dependent. This large DM is also what distinguishes them from individual pulses from pulsars, as FRB pulses will have a significantly higher DM than their pulsar counterparts, which are Galactic objects as their pulses are too faint to observe from other galaxies. The minimum DM required for a pulse to originate from outside our Galaxy is dependent on the amount of interstellar matter the pulse has to travel through to reach our telescope, which differs for each direction as a result of our position in the Milky Way. In the case of low Galactic latitude lines of sight, there is more foreground Milky Way DM, while there is less at high latitude lines of sight. There are models for the Galactic distribution of free electrons, such as NE2001 [7] and the newer YMW16 [19], as well as models for the contribution from the intergalactic medium [11]. Using these models, the approximate distance of an FRB can be found.

FRBs are amongst the most energetic astrophysical events, emitting energies within a few orders of magnitude less than supernovae [15] in a much shorter time span. Because of their intense energy and the immense temperature that would be required for such a burst, they are not thermal in nature, but originate from a coherent emission process instead [16].

While their origin is not yet known, many theories connect FRBs to neutron stars [16]. This is because of their short durations which suggest a compact source, and due to their high temperature brightness, where neutron stars with high magnetic fields could generate intense bursts of radio waves. These features align well with what we observe from neutron stars, making them strong candidates for the sources of FRBs.

A common obstacle encountered in the hunt for FRBs is radio frequency interference (RFI). This terrestrial interference originates from a wide variety of sources, such as electronic devices, satellites, lightning or passing cars. RFI can

induce false positives in datasets by creating bright patches of data that can mislead FRB detection algorithms. RFI exist as narrow-banded interference, where a few frequency channels are, often continuously, affected. The other type of RFI is broad-banded interference, where RFI is present in all frequency channels, often in smaller patches of data.

RFI signals are much more common than FRBs, and classifying candidate astrophysical signals is a time-consuming task. Methods to mitigate effects from RFI differ per type of RFI. For narrow-banded RFI, the affected channels are often excised (‘masked’) from the data. For broad-banded RFI, we can employ filters, which are algorithms that can remove individual pixels or small patches of data that are affected by RFI. A filter that we will use throughout this paper is the Z-dot filter [12].

Some RFI have a similar impulsive nature and timescale as FRBs, but since they originate from Earth, the DM of such FRB-mimicking RFI is usually zero. Therefore, non-zero DM is used as another metric to verify the astrophysical origin of a detection.

While most FRBs are detected on millisecond-timescales, they have been detected with durations ranging from sub-microseconds [18] up to a few seconds [4]. At extremely short timescales especially, detecting FRBs requires sensitive telescopes with a high time resolution, and thus efficient algorithms are necessary to find them in the ever increasing flow of astronomical data.

## 1.1 TransientX

TransientX [13] is a software tool for detecting FRBs. Released in early 2024, it aims to offer an efficient pipeline for finding single pulses in astronomical data. Unlike Heimdall [5]<sup>1</sup>, which utilises GPU-based acceleration for finding FRBs, TransientX exclusively uses the available CPU(s) for data processing.

TransientX also includes a tool named ‘replot’, which can be run on candidates found by TransientX to re-evaluate them and to refine their properties. If these refined properties indicate the burst is RFI, it can be filtered from the candidate pool, greatly reducing the amount of candidates that need further evaluation.

## 1.2 Motivation

The study of FRBs is a fascinating scientific field, as FRBs present us a unique view of extreme astrophysical sources at extragalactic distances. FRBs also allow us to probe the interstellar and intergalactic media that the radio waves travel through before they reach the telescope, allowing us to constrain properties on dark matter and dark energy [20], map the distribution of baryonic

---

<sup>1</sup>While the URL in the original paper no longer exists, the Heimdall repository can be found on the official Sourceforge page at <https://sourceforge.net/projects/heimdall-astro>

matter in intergalactic space [9], and constrain properties of intergalactic magnetic fields [3] [14].

With data from telescopes increasing in quantity and resolution, we currently obtain much more data than we can reliably process. We need more advanced and optimised data processing techniques to find FRBs in this torrent of data. A milestone in the field of FRB studies would be the ability to process data as quickly as we can obtain it (i.e. real-time analysis) on affordable hardware, especially if this can be achieved on data with a high time-resolution where sample lengths are significantly less than 1 millisecond.

The recent release of TransientX is promising for the study of FRBs and we aim to benchmark its performance and accuracy on a large, high-time-resolution dataset on our available computing hardware. We also present a number of parameters that can be used for various purposes, such as finding millisecond-duration bursts and finding microsecond-duration bursts. We will briefly touch upon a future roadmap towards real-time processing, where data can be analysed as quickly as, or quicker than, it can be captured by a telescope, which would result in the ability to process all incoming data efficiently.

## 2 Observations

### 2.1 Environment

We run TransientX in a Singularity environment, which allows us to run software independently from the rest of the system. Singularity is a container platform similar to Docker, which allows to run software in an isolated and containerised environment, with its own installed packages. This allows for flexibility in a restricted production environment, easy management of different versions of software, and allows for better reproducibility as one can ensure the results are obtained using an exact, specific set of software and associated libraries.

All data processing was performed on a dedicated processing machine, *ebur*, situated at the Onsala Space Observatory in Sweden. It has a dual-socket Intel Xeon E5-2640 v4, with 20 physical cores and 40 virtual cores in total, and features 128 GB of DDR4 RAM. The host OS is Linux Mint 21.2. The storage that is used to load the datasets is a RAID5 consisting of 5 HGST Ultrastar 7K6000 HDDs, each of which has a capacity of 6 TB, totalling 24 TB of storage<sup>2</sup>. This storage benchmarked at a sustained read speed of 730 MB/s.

---

<sup>2</sup>Not 30 TB as one might expect, since RAID5 uses a combined total of one disk's capacity for redundancy

## 2.2 Datasets

### 2.2.1 Trial datasets

We used two datasets to test that TransientX was functional on our system and could detect known FRBs. Both datasets were recorded at the Westerbork Synthesis Radio Telescope located in the Netherlands.

**The first dataset** contained a single FRB, FRB20201124A. It was recorded on January 26th, 2022 at 19:56:50 UTC and the dataset lasts 2.9 seconds. The time resolution is 16 microseconds, and the frequency range is 1259.5 MHz to 1387.5 MHz, with a channel width of 0.0625 MHz, making for 2048 channels.

**The second dataset** was much larger, and was recorded two days later, on January 28th, 2022 at 17:24:16 UTC. This dataset had a duration of 869.5 seconds, with a time resolution of 256 microseconds. The frequency range spanned 300 MHz to 364 MHz, divided in 16384 channels, making for a frequency resolution of 3.91 kHz. The FRB in this dataset is FRB20220912A.

### 2.2.2 High time resolution dataset

Lastly, we ran TransientX on a dataset with a high time resolution. This dataset was recorded in Germany using the 100-m Effelsberg radio telescope on February 20th, 2024. The data consists of a million time samples per second, making for a time-resolution of  $1\mu\text{s}$ . The lowest frequency channel is 1265.125 MHz, while the highest channel is 1509.875 MHz.

This dataset was divided into two parts: a longer section that we will use for the benchmarks, and a shorter one containing only the FRB. Initially, we intended to use a single dataset for both benchmarks and FRB detection. However, the dataset was found to be cut before the FRB occurred, so an extra dataset containing only the FRB was made.

The benchmark section was 224.256 seconds long and started on 09:20:23 UTC. The FRB section was 2.56 seconds long and started at 09:29:06 UTC. The benchmark section has 1024 0.25-MHz frequency channels per time-step, while the FRB section has 512 0.5-MHz frequency channels per time-step.

## 3 Analysis

### 3.1 Trial Runs

Although the occurrence time and properties of the FRB were known, the trials were approached blindly, to see if the signal could be found by TransientX with a non-specific parameter set. For these trial runs we used no downsampling, a standard DM start value of zero, using 1024 steps with a step size of  $2\text{ cm}^{-3}\text{ pc}$ ,

which resulted in a highest DM trial of  $2048 \text{ cm}^{-3} \text{ pc}$ . We used the default signal-to-noise threshold of 7, with the Z-dot filter, which removes pulses with a DM of zero  $\text{cm}^{-3} \text{ pc}$ , and the IQR option, which re-calculates the mean and variance for each data chunk and is suitable for data with multiple bright bursts. Since we are interested in FRBs in the millisecond-timerange width, we used a minimum width parameter of 0.5 ms and a maximum width parameter of 100 ms. We assigned TransientX ten threads on the host machine.

After the initial pass, we re-evaluated the candidates using replot. For this purpose, we used 20 threads and the Z-dot filter. We passed the ‘clean’ parameter and a DM cutoff of 25, which would result in any candidates with a DM under  $25 \text{ cm}^{-3} \text{ pc}$  to be dropped. By default, the signal-to-noise cutoff is 7, meaning that if the optimised signal-to-noise ratio is under 7, the re-evaluated candidate would also be dropped.

### 3.2 High-time-resolution run

For the high-time-resolution run, we adopted a similar approach as we did for the trial runs, where we did not take known properties into account when searching for the FRB. To ensure a feasible runtime despite the high resolution, we used a DM step size of  $8 \text{ cm}^{-3} \text{ pc}$  with 256 DM steps, where we intend to compensate for these inaccurate parameters using replot. All other parameters, including replot parameters, were identical to the trial runs.

### 3.3 TransientX Benchmarks

To accelerate the benchmarks we limited the data that was processed to just over 24 seconds of data, which is just under 11 percent of the total high-resolution dataset that was available to us. During the benchmark runs, we observed the activity on the machine to confirm that no other users were running processes that might have increased the load on the machine and interfered with the results.

We gathered the benchmark results by altering a single variable over a pre-determined range, and using Linux’s ‘time’ command to measure the duration of the run. We also used Linux’s ‘htop’ to measure RAM usage. After each run, we computed the relative amount that the run was slower compared to real-time as a factor using the equation

$$\text{slowdown factor} = \frac{\text{real runtime in seconds}}{24.26 \text{ s}} . \quad (1)$$

If this value reaches one or under, that means the run completed faster than the duration in which the data was recorded, achieving or surpassing real-time analysis.

We also determined the benefit of using multiple CPU threads for each run by determining the amount that the cumulative runtime of all threads combined was higher than the real runtime. We computed this using the equation

$$\text{multi-threading benefit factor} = \frac{\text{cumulative runtime of all threads in seconds}}{\text{real runtime in seconds}}. \quad (2)$$

This metric can grant us some crude insights into the benefit of using multiple threads, as well as identify any bottlenecks. It is important to note that this is the relative benefit, dependent on the total runtime. It is possible that, for differing runs, the relative benefit is lower but the absolute benefit is higher if the runtime of the second run was higher.

Aside from the aforementioned metrics, we also kept track of the RAM usage and the amount of candidates found after each run. Importantly, since the benchmark data contains no known FRBs, the candidates found are all noise or RFI.

## 4 Results

### 4.1 Trial datasets

#### 4.1.1 Dataset 1

Running TransientX and replot on the dataset yields the FRB shown in Figure 1. In total, 101 candidates were found during the initial pass. After running replot with the parameters described in section 3.1, only 9 candidates remained, out of which the FRB was identified by eye. The majority of rejected candidates were detections of the same pulse, but with an incorrect de-dispersion.

#### 4.1.2 Dataset 2

Running TransientX and replot on the dataset yields the FRB shown in Figure 2. The initial pass of this dataset found 966 candidates. Replot removed all but 9 of these candidates based on the thresholds described earlier in section 3.1, out of which the FRB was identified by eye. Most of the rejected candidates were detections of noise and RFI.

### 4.2 High-time-resolution dataset

The FRB found in the high-time-resolution dataset can be seen in Figure 3. The initial pass detected 200 candidates, out of which 2 remained after replot's second pass based on the thresholds described earlier in section 3.1. Out of the candidates optimised by replot, the FRB was identified by eye.

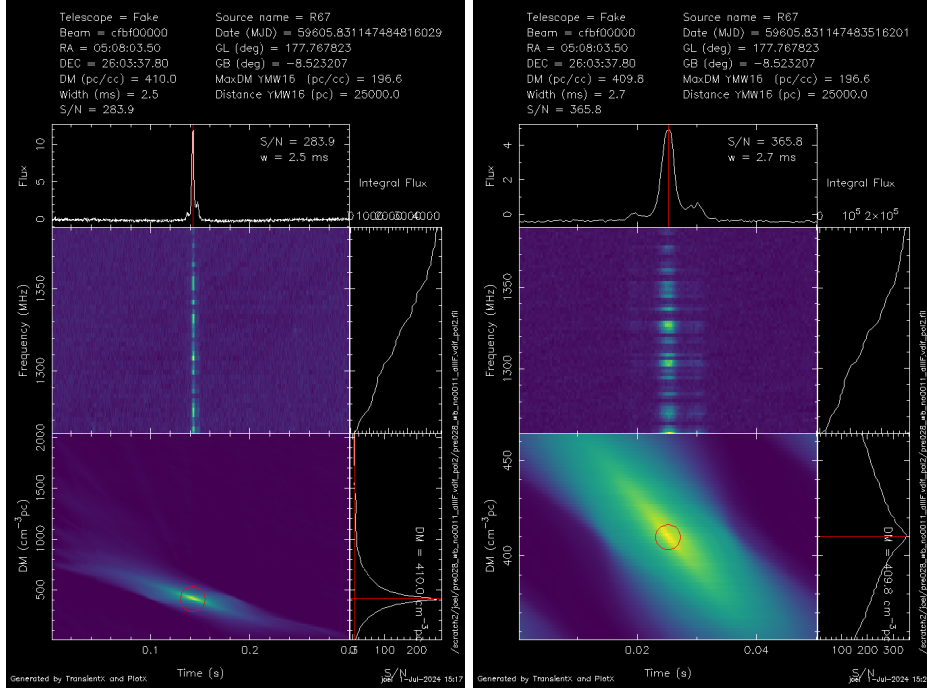


Figure 1: The FRB detected in the dataset pre028\_wb\_no0011\_allIF.vdif\_pol2. On the left is the original detection by the first pass from TransientX, and on the right is the optimised detection from replot. The initial detection assigned a signal-to-noise ratio of 283.9, a DM of  $410.0 \text{ cm}^{-3} \text{ pc}$  and a width of 2.5 ms, while replot optimised the signal-to-noise ratio to 365.8, the DM to  $409.8 \text{ cm}^{-3} \text{ pc}$  and the width to 2.7 ms. A distinct vertical line is visible on the frequency-time plot, while the DM-time plot shows the desired bowtie-like shape. The DM is sufficiently high to be of extragalactic origin.

### 4.3 TransientX Benchmarks

#### 4.3.1 Time-downsampling benchmarks

**Millisecond range** The results of varying the amount of downsampling in time and measuring the runtime can be observed in Figures 4, 5 and 6. These benchmarks focused on searches in the millisecond range.

**Microsecond range** These benchmarks focused on data with little RFI, performing searches in the microsecond width-range. The results can be found in Figure 7.

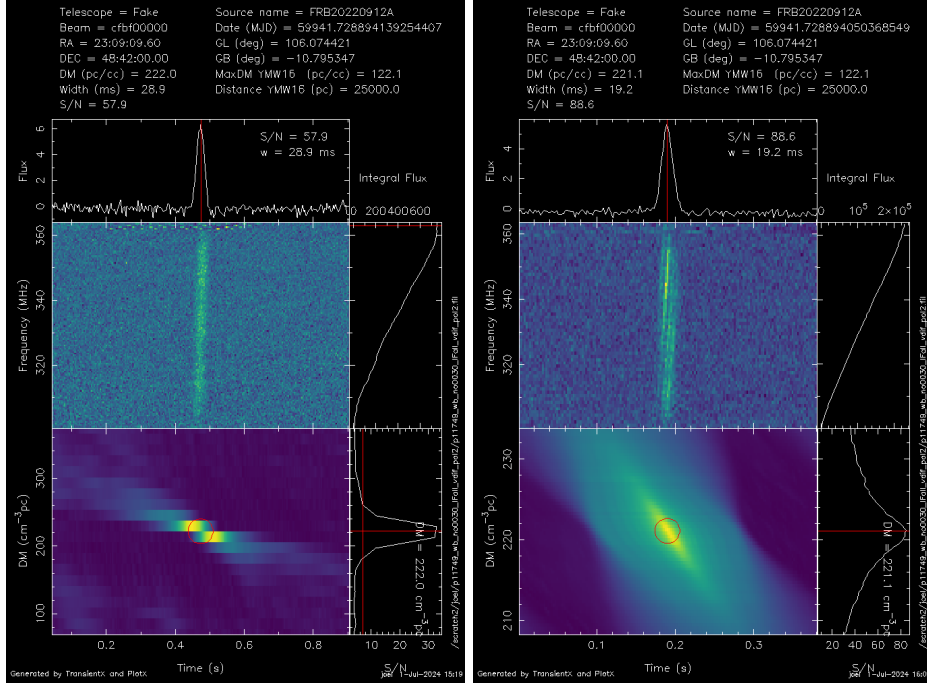


Figure 2: The FRB detected in the dataset p11749\_wb\_no0030\_IFall\_vdif\_pol2. On the left is the original detection by the first pass from TransientX, and on the right is the optimised detection from replot. The initial detection assigned a signal-to-noise ratio of 57.9, a DM of  $222.0 \text{ cm}^{-3} \text{ pc}$  and a width of 28.9 ms, while replot optimised the signal-to-noise ratio to 88.6, the DM to  $221.1 \text{ cm}^{-3} \text{ pc}$  and the width to 19.2 ms. A distinct vertical line is visible on the frequency-time plot, while the DM-time plot shows the desired bowtie-like shape. The DM is sufficiently high to be of extragalactic origin.

#### 4.3.2 S/N threshold benchmarks

To test the impact of clustering, we increased the signal-to-noise ratio required for generating a candidate. We ran this over a range from 7 to 19 in a step size of 2. The data point with a threshold of 9 is missing; this specific configuration caused a TransientX crash and thus was not included. The initial RAM usage was 67.3 GB for each run. We fixed the time downsampling factor at a value of 8. The results of this benchmark are shown in Figure 8.

#### 4.3.3 DM step count benchmarks

To test the effect of the number of DM steps, we ran a benchmark where we set the time downsampling to a fixed value. We then ran runs with an increasing amount of DM steps, changing the step size as needed to keep the same DM

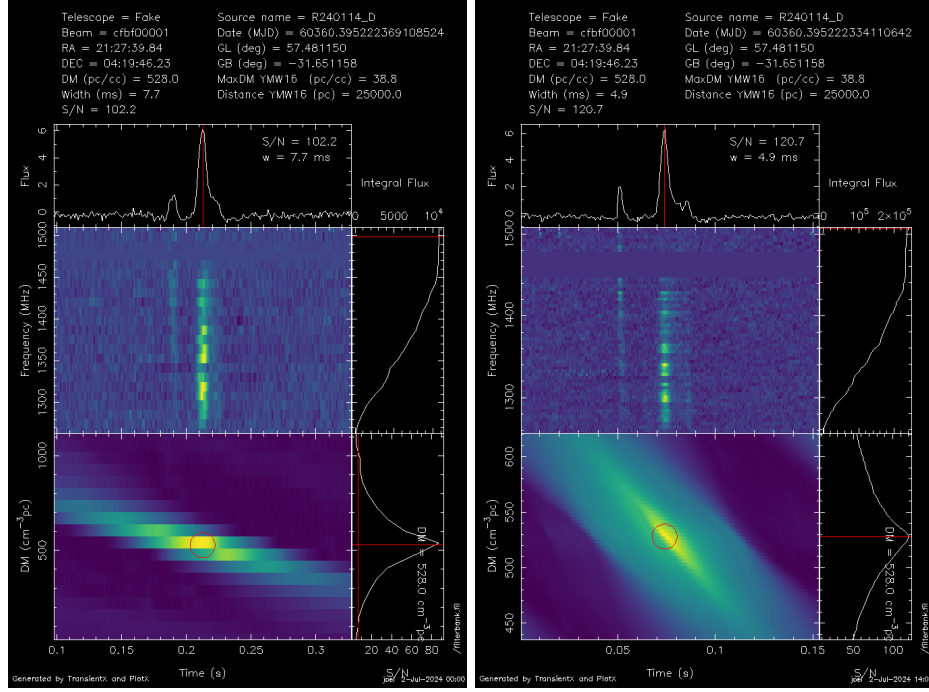


Figure 3: The FRB detected in the high-time-resolution dataset. On the left is the original detection by the first pass from TransientX, and on the right is the optimised detection from replot. The initial detection assigned a signal-to-noise ratio of 102.2, a DM of 528.0 cm<sup>-3</sup> pc and a width of 7.7 ms, while replot optimised the signal-to-noise ratio to 120.7, kept the DM at 528.0 cm<sup>-3</sup> pc, and optimised the width to 4.3 ms.

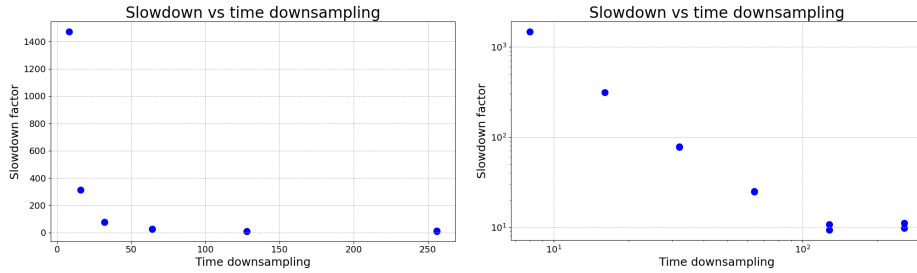


Figure 4: The slowdown in terms of real-time speed plotted against the level of downsampling used. On the left is a plot with linear axes, and on the right a plot with logarithmic axes. Higher slowdown values indicate a longer runtime. The logarithmic plot shows a predictable pattern until a plateau is hit where the runtime does not decrease anymore, even with higher downsampling.

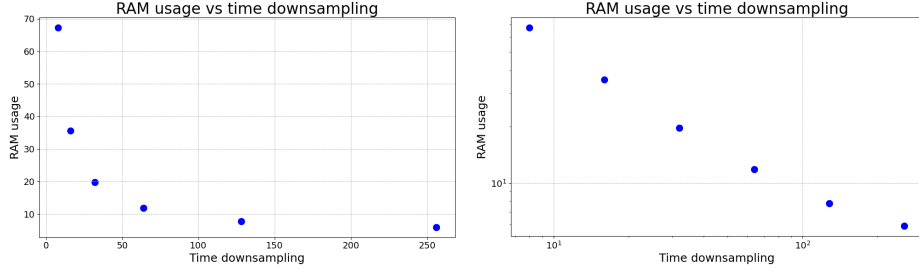


Figure 5: TransientX’s RAM usage plotted against the level of downsampling used. On the left is a plot with linear axes, and on the right a plot with logarithmic axes. The logarithmic plot shows a predictable pattern that seems to slightly plateau as the time downsampling is increased.

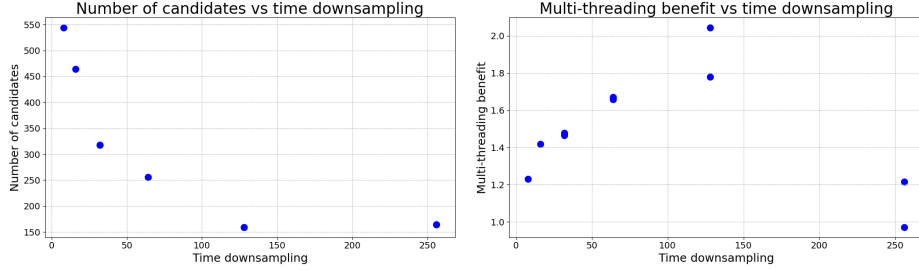


Figure 6: On the left: the amount of candidates remaining after the clustering process. On the right: the amount of relative benefit an increase in available threads brings.

range.

**Time downsampling of 8, ms search** For this benchmark, we used a time downsampling factor of 8 to test the effect of the amount of DM steps when time resolution is high. The results of this benchmark are shown in Figure 9.

**Time downsampling of 8,  $\mu$ s search** We also tested a run in the microsecond time-range, to measure the effects this would have on runtime. The results are shown in Figure 10.

**Time downsampling of 32, ms search** Here we used a time downsampling factor of 32, to test the effect of the amount of DM steps when time resolution is lower. The results are shown in Figure 11.

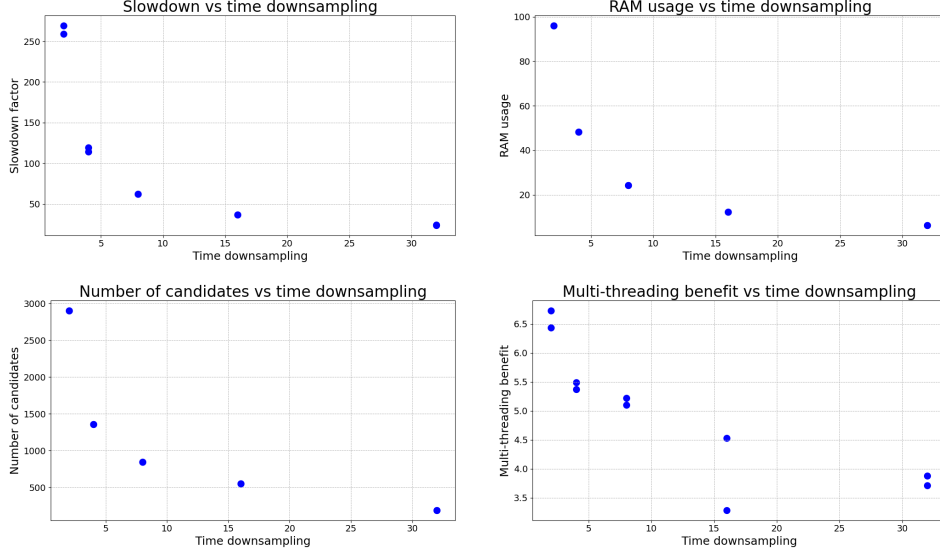


Figure 7: The results of the benchmarks in the microsecond width-range varying the time downsampling. On the upper left plot, the slowdown factor is plotted against the time downsampling factor. The upper-right plot shows the measured initial RAM usage of each run. The lower-left plot shows the amount of candidates resulting from each run, and the lower-right plot shows the relative benefit gained from multi-threaded processing.

## 5 Discussion

### 5.1 Trial runs and high-time-resolution run

The FRB detections from the trial runs, as seen in Figures 1 and 2, show that the FRBs were detected with measured parameters that are consistent with the values in the literature. This confirms that TransientX is functional on our machine and that our chosen parameter set is effective at finding all known FRBs in datasets, which have previously been detected using another pipeline based on Heimdall. Likewise, running TransientX on the data section where the FRB was included, the FRB was detected with parameters consistent with values in literature. The results can be seen in Figure 3.

### 5.2 Benchmark results

The benchmarks show that TransientX’s run duration is reliable and that different choices of parameters can result in a focus on processing speed and processing accuracy, with a clear inverse correlation between these two. Important to note is that the measured RAM usage in the benchmarks is the initial RAM that TransientX reserves. During the run, this value will fluctuate upwards, but

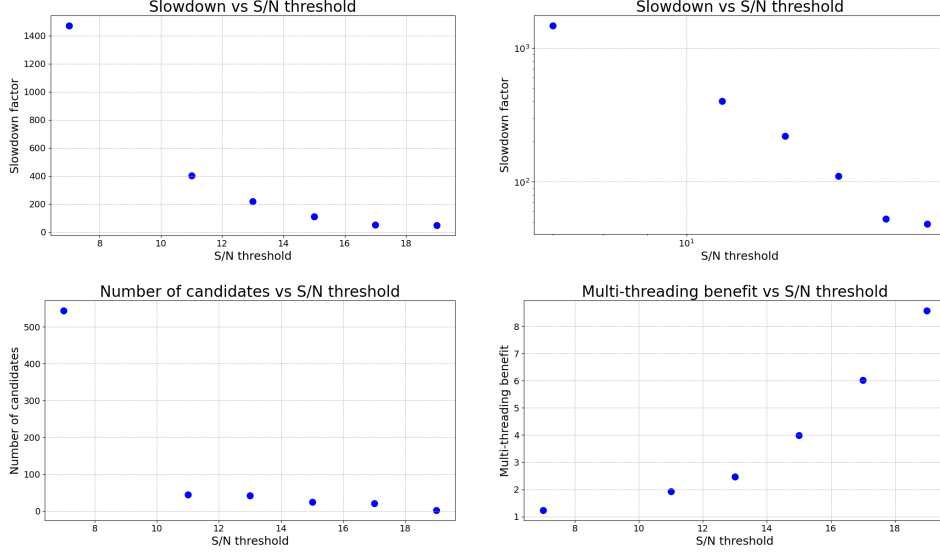


Figure 8: The results of the S/N threshold benchmarks where we varied the threshold from 7 to 19 with a fixed time downsampling of 8. On the upper-left plot, the slowdown factor is plotted against the signal-to-noise threshold used, and the upper-right shows the same but with logarithmic axes, showing the correlation between a higher threshold and a lower runtime more clearly. The lower-left plot shows the amount of candidates resulting from each run, and the lower-right plot shows the relative benefit gained from the usage of multiple threads. The initial measured RAM usage was consistently 67.3 GB, and was not included in this plot.

as the fluctuations appear less than 20 percent, this measured value should still provide a representative estimate of expected RAM usage.

### 5.3 TransientX stability

We originally intended to run every benchmark on both a cleaner patch of data, where the data is well described as Gaussian noise, and on a patch of data with more RFI present. However, for some parameter sets and data chunks, TransientX seemed prone to experience segmentation faults. These crashes occur consistently at specific parameter sets, and changing the parameters slightly or running on a different dataset can prevent the crash. We omitted the high-RFI benchmarks, as the amount of data points we could gather were insufficient to draw conclusions.

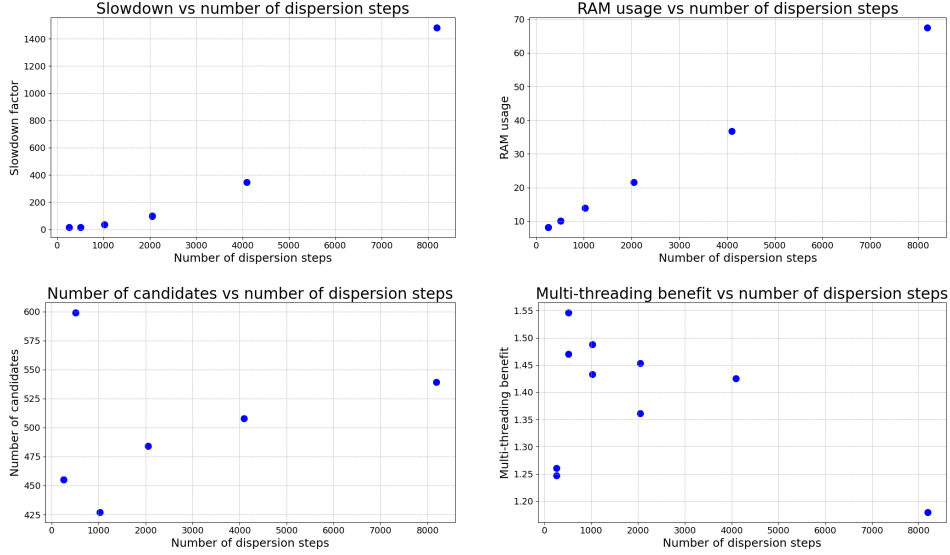


Figure 9: The results of the benchmarks with varying number of DM steps on a fixed time downsampling factor of 8. In the upper-left plot, the slowdown factor is plotted against the number of DM steps. The upper-right plot shows the measured initial RAM usage of each run. The lower-left plot shows the amount of candidates resulting from each run, and the lower-right plot shows the relative benefit gained from multi-threaded processing.

## 5.4 Bottlenecks

In our experience, data processing happens in three phases. For each chunk of data, by default one second, TransientX loads the data from storage, performs de-dispersion and analysis steps, and clusters the results to prevent duplicates. These phases seem to happen serially, where the next phase cannot start as long as the current is still ongoing. Each of these phases will have their own bottleneck.

The multi-threading benefit benchmarks shown in Figures 6 to 11 show how bottleneck causes shift as the measured parameter changes. In Figure 6, there is a clear peak visible, and the graph illustrates all three phases of bottlenecks. On the low time downsampling parameters, the benefit of multi-threading is low, since most of the time will be spent clustering and not during the de-dispersion and matched filtering steps. At high time downsampling parameters, the benefit of multi-threading is also low, as the main bottleneck here is storage read speed. At the peak, at a time downsampling of 128, the multi-threading benefit is the highest as the severity of the other bottlenecks is smaller than at the extreme values. The other figures show similar patterns of shifting bottlenecks, although the peaks are less clearly visible or are not present.

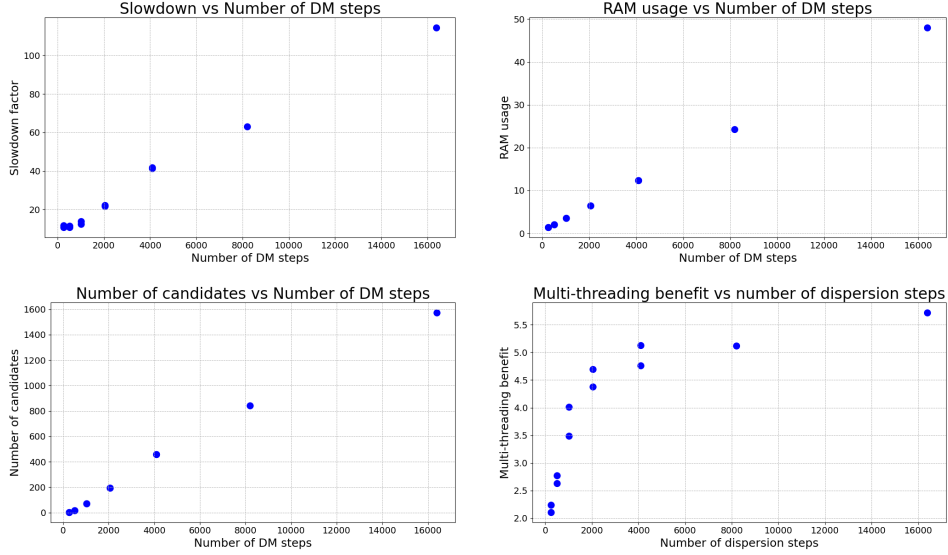


Figure 10: The results of the benchmarks with varying number of DM steps on a fixed time downsampling factor of 8. In the upper-left plot, the slowdown factor is plotted against the number of DM steps. The upper-right plot shows the measured initial RAM usage of each run. The lower-left plot shows the amount of candidates resulting from each run, and the lower-right plot shows the relative benefit gained from multi-threaded processing.

#### 5.4.1 Causes of bottlenecks

During the data loading phase, the performance of TransientX largely depends on storage read performance. While it is technically possible that this phase is slowed by single-thread performance, the CPU would have to be disproportionately slow compared to storage speed, which is unlikely in any normal setup. If TransientX is run on the same data repeatedly, speed could improve as the system may cache the raw data to RAM, but this is once again unlikely as filterbank files are usually much larger than the available RAM.

Pre-processing and processing steps are run on all available threads, and benefit from increased threads, increased core frequency and increased instructions per cycle. This phase is usually relatively quick, and from the benchmarks we can conclude that this is not the main bottleneck.

The clustering phase is, especially when many candidates have been found, usually the slowest phase in a chunk of processing. This process is single-threaded, and benefits only from increased core frequency and increased instructions per cycle.

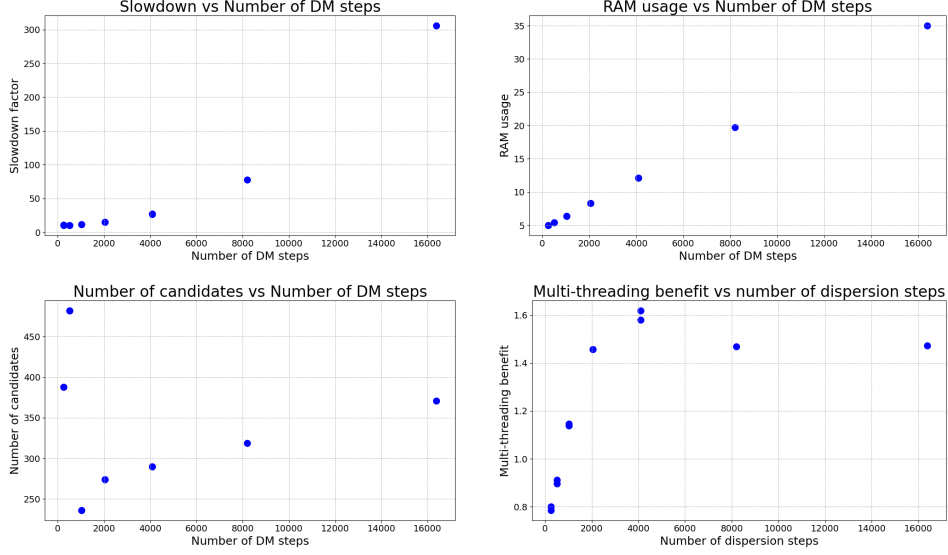


Figure 11: The results of the benchmarks with varying number of DM steps for a fixed time downsampling factor of 32. In the upper-left plot, the slowdown factor is plotted against the number of DM steps. The upper-right plot shows the measured initial RAM usage of each run. The lower-left plot shows the amount of candidates resulting from each run, and the lower-right plot shows the relative benefit gained from multi-threaded processing.

The final TransientX bottleneck is the RAM usage, which, as seen on the benchmarks, can quickly grow until the system runs out of RAM. More available RAM means more runs can be done simultaneously, or the runs that are being performed can be of a higher resolution.

#### 5.4.2 Mitigation of bottlenecks

We have a number of options at our disposal to minimise the impact of these aforementioned bottlenecks. Firstly, we can reduce the block size from the default of one second to reduce RAM usage. This is especially useful when searching for bursts with a microsecond width. Furthermore, increasing the time downsampling or frequency downsampling will reduce RAM usage as well as the amount of data to process and the number of results to cluster. This results in a large speedup, but if time and frequency resolution are too coarse, low-width or high-DM FRBs may be missed entirely. A method to reduce the time spent clustering is to increase the S/N threshold, as this will result in fewer candidates to cluster, but this may miss bursts with a low brightness. The clustering phase can also be sped up using a lower DBSCAN radius, although this may lead to multiple redundant detections of the same pulse. Lastly, another effective method of accelerating the clustering process is by increasing the DM step size,

although using a too high DM step size can cause bursts to be missed entirely, and the results of TransientX’s first pass will be less accurate. The impact on accuracy can be limited by replot, as it will optimise the DM of the detected candidates, but the minimum step size resulting from the desired minimum time resolution should not be exceeded.

### 5.4.3 Potential TransientX improvements

As the clustering bottleneck is largely a problem within TransientX itself, we hope future improvements will be made in this area. A potential method to reduce the impact of the bottleneck is to continue processing the upcoming chunk of data during the clustering of the previous, reserving an available thread for this process and continuing data processing with one less thread. Assuming DBSCAN only takes properties of the candidates and not any raw data, this would not result in excessive RAM usage, as the data can be safely unloaded and the next chunk can be loaded. If  $N$  is the amount of threads that can be used by TransientX as specified using the `-t` parameter, and  $M$  is the amount of lingering clustering processes, there would be  $N - M$  threads available for de-dispersion and matched filtering. A limit could be set such that  $N - M \geq 2$ , always leaving at least two threads for these data processing steps. When reaching this limit, TransientX could then halt until a lingering clustering process has finished. However, this method has its own downsides, such as highly increased code complexity and losing the guarantee that all outputted candidates are sorted by occurrence time.

More potential options would include using a different, faster clustering algorithm instead of DBSCAN, which might be a bit less accurate but faster, or employ either and let the user choose the faster algorithm using a command-line argument. A potential candidate could be HDBSCAN. It is also possible to delay clustering until after all chunks have finished their processing steps, and run them all in parallel. This prevents complex core management during the run, but has the downside that no results are generated until the run is finished. It also introduces the need to temporarily store the data until clustering. Lastly, the DBSCAN algorithm could be modified such that the candidates are divided over multiple threads by occurrence time. If two candidates are separated further than the DBSCAN radius, those cannot be assigned the same cluster, so we can distribute the workload.

#### 5.4.4 Search strategies

When searching for candidates using TransientX, different goals require different strategies. It is advisable to use different parameters for searches targeting FRBs with a millisecond-width as opposed to searches targeting FRBs with a microsecond-width. When performing blind searches for FRBs in both the millisecond width range and microsecond width range, both these parameter sets should be used in separate runs. In the following paragraphs, we will

discuss recommended parameter sets for these different cases.

**Millisecond searches** For searching FRBs in the millisecond width range, the default parameters are a good starting point. Time downsampling can be set to any reasonable value, while still maintaining a time resolution smaller than the lower width boundary, and the signal-to-noise ratio should be adjusted accordingly. If the data is of sufficient frequency resolution, slight frequency downsampling can also be employed. The minimum width should be set to 0.0005 (0.5 ms), because the microsecond search strategy will be used for FRBs of lower widths. The maximum width can be set at any desired value, and optionally can be combined with the ‘drop’ parameter, which removes any candidates exceeding the maximum width. It is, however, recommended to leave this option off, as TransientX can sometimes select a candidate with an incorrect DM causing the FRB to smear in time, making it appear wider than it is. Since we use replot - which can correct such inaccuracies - as second pass, it may not be desirable to drop these candidates.

For a blind search where no FRBs with known properties are expected, the DM range should start at  $0 \text{ cm}^{-3} \text{ pc}$ , as this will be useful to quickly detect RFI bursts. When searching for repeat bursts from an FRB with a known DM, the DM range can be narrowed down to those known properties to greatly increase speed. If replot is used as a second pass to optimise the detected candidates, the DM step size can be increased. We consistently found good results using this methods with DM step sizes up to  $8 \text{ cm}^{-3} \text{ pc}$  on the datasets that contained an FRB, but if the DM step size is too large, bursts may be missed entirely. The number of DM steps can be set arbitrarily such that the desired upper DM value is achieved.

It may be recommended to set a slight overlap ratio of 0.1, to avoid missing a burst that might be right on the boundary of a data chunk, although future research will have to confirm this advice. The Z-dot filter should also be used to reduce the impact of RFI, and the DBSCAN radius can be increased from the default of 1 ms to avoid duplicate detections, although this comes with a sizeable impact on processing speed when candidates are detected.

After the initial run has finished, the detected candidates can be further analysed using replot. Here, it is recommended to decrease downsampling values as much as possible without excessively sacrificing processing speed. The ‘clean’ parameter should be used to exclude candidates that do not have FRB-like properties. The specific thresholds for these cleaning parameters can be set as desired, and can be found with replot’s help command.

**Microsecond searches** For searching within the microsecond time range, some parameters have to be changed to improve speed and detection perfor-

mance. It is recommended to decrease time downsampling by as much as possible and to increase the signal-to-noise threshold because of the increased amount of trials. To decrease RAM usage, the block size should be lowered to a reasonable value such as 0.1 seconds. The DBSCAN range should also be lowered to a smaller value such as 0.01 ms. The minimum width can be set to the time resolution (taking time downsampling into account), while the maximum width should be set to 1 ms, combined with the drop parameter, to drop any candidates over this width limit, as any wider candidates should have been found by the millisecond run. Aside from these changes, the recommended parameters from the millisecond run setup should be used.

**Real-time processing** The ability to process data in real-time is highly dependent on the properties of the data. Theoretically, assuming storage speed is not a bottleneck, data with a sufficiently low resolution can be processed as quickly as it can be captured from a telescope. However, in practice, the clustering bottleneck is too large to achieve this, especially if RFI is present in the data.

## 5.5 Effective real-world usage of TransientX

While this paper has focused on benchmarking TransientX, more experimentation should be conducted on how to effectively apply the use of TransientX in real-world use cases. Since TransientX is bottlenecked to a single thread for most of the duration of the run, multiple runs could be conducted simultaneously if RAM capacity allows. One option is to run the searches for millisecond and microsecond timescales simultaneously, but it is also possible to split runs using TransientX’s jump parameter (-j), where a time in seconds can be skipped at the start and end of the filterbank file. As long as the machine does not run out of memory and the storage bottleneck is minor compared to the clustering bottleneck, performing runs simultaneously will avoid the machine running idle for most of the time.

It is advisable to make at least two passes through the data using the previously described parameter sets, such that every width range can be analysed effectively and efficiently. If a single run were to be performed over all width ranges, the parameter set used for this run would be highly inefficient for FRB width trials on either side of the extreme, as some parameters such as the DBSCAN width and block size would not be suitable for these trial widths.

## 5.6 Future research

Future research could focus on improving and extending the benchmarks presented in this paper. One such method would be to vary both the parameter the benchmark is aimed at, along with the signal-to-noise threshold, such that a fixed certainty is maintained as opposed to a fixed signal-to-noise ratio. Furthermore, future benchmarks could inspect potential speed gains when the

input filterbank file has been pre-downsampled instead of performing the downsampling on-the-fly. This might result in a speedup, since the storage bottleneck decreases, along with the overhead for downsampling during data loading. Lastly, decreasing the amount of bits per data point has the potential to result in a speed increase too, since less data has to be loaded and stored in RAM. However, astrophysical signals in the data might not be as well described or suffer from artifacts as a result of this operation, and the effects of this on speed and accuracy should be benchmarked to measure the feasibility of this approach.

Since false positives as a result of RFI are a contributing factor to the clustering bottleneck, preventing these RFI from causing detections at all would help considerably. In this paper, we used the Z-dot filter, but experimentation should be conducted to measure the effect of other filtering algorithms or combinations thereof.

If multiple runs are to be conducted simultaneously in production, it is desirable to have an accurate metric of the expected RAM usage. When benchmarking on different parameters, a function should be able to be fit to the RAM usage. If this is repeated for every parameter, expected RAM usage can be computed for an arbitrary set of parameters.

Future work could also focus on completing the end-to-end pipeline by integrating machine learning (ML) algorithms for automatic candidate classification. Existing ML algorithms include FETCH [2] [1], for which we wrote a candidate conversion tool<sup>3</sup>, which converts the TransientX candidate file to a FETCH-compatible format. However, a ML model trained directly on TransientX's data might perform classification quicker as it eliminates any intermediate conversion and processing steps.

## 6 Conclusions

During the trial runs on smaller datasets, TransientX found all FRBs that were known to exist in these datasets. This confirms that TransientX is working as intended with the parameter set that was used.

While no FRB was present in the initial high-time-resolution dataset, we did find it in the separate dataset using the same chosen initial parameters. The properties of the found FRB are in agreement with the properties that were already known, so we can confirm that this is the same FRB we were expecting to find.

We suggested parameter sets to use in millisecond searches and microsecond searches along with general advice, which should provide a good starting point

---

<sup>3</sup><https://github.com/JoelvanIngen/TransientX-Tools>

for both further research and real-world application of TransientX.

We found TransientX’s performance to be very promising in some areas, while being limited in others. Pre-processing and processing steps proved very fast, with the slower clustering phase in a strong contrast. We suggested multiple techniques to work around these limitations, as well as some potential solutions that the TransientX authors could implement in the future.

## 7 Acknowledgements

Firstly, I would like to thank both of my supervisors of this project, for their continued assistance and invaluable feedback throughout.

I would also like to extend a thank you to my friends and family members who have supported me throughout the project and proof-read my paper during the writing process.

Finally, I would like to thank the members of the research group AstroFlash for allowing me to be a part of this fascinating research.

## References

- [1] Devansh Agarwal and Kshitij Aggarwal. devanshkv/fetch: Software release with the manuscript, June 2020.
- [2] Devansh Agarwal, Kshitij Aggarwal, Sarah Burke-Spolaor, Duncan R Lorimer, and Nathaniel Garver-Daniels. FETCH: A deep-learning based classifier for fast transient classification. *Monthly Notices of the Royal Astronomical Society*, June 2020.
- [3] Takuya Akahori, Dongsu Ryu, and B. M. Gaensler. Fast radio bursts as probes of magnetic fields in the intergalactic medium. *The Astrophysical Journal*, 824(2):105, June 2016.
- [4] Bridget C. Andersen, Kevin Bandura, Mohit Bhardwaj, P. J. Boyle, Charanjot Brar, Daniela Breitman, Tomas Cassanelli, Shami Chatterjee, Pragma Chawla, Jean-François Cliche, Davor Cubranic, Alice P. Curtin, Meiling Deng, Matt Dobbs, Fengqiu Adam Dong, Emmanuel Fonseca, B. M. Gaensler, Utkarsh Giri, Deborah C. Good, Alex S. Hill, Alexander Josephy, J. F. Kaczmarek, Zarif Kader, Joseph Kania, Victoria M. Kaspi, Calvin Leung, D. Z. Li, Hsiu-Hsien Lin, Kiyoshi W. Masui, Ryan Mckinven, Juan Mena-Parra, Marcus Merryfield, B. W. Meyers, D. Michilli, Arun Naidu, Laura Newburgh, C. Ng, Anna Ordog, Chitrang Patel, Aaron B. Pearlman, Ue-Li Pen, Emily Petroff, Ziggy Pleunis, Masoud Rafiei-Ravandi, Mubdi Rahman, Scott Ransom, Andre Renard, Pranav Sanghavi, Paul Scholz, J. Richard Shaw, Kaitlyn Shin, Seth R.

- Siegel, Saurabh Singh, Kendrick Smith, Ingrid Stairs, Chia Min Tan, Shriharsh P. Tendulkar, Keith Vanderlinde, D. V. Wiebe, Dallas Wulf, and Andrew Zwaniga. Sub-second periodicity in a fast radio burst. *Nature*, 607(7918):256–259, July 2022.
- [5] B. R. Barsdell, M. Bailes, D. G. Barnes, and C. J. Fluke. Accelerating incoherent dedispersion: Accelerating incoherent dedispersion. *Monthly Notices of the Royal Astronomical Society*, 422(1):379–392, March 2012.
- [6] The CHIME/FRB Collaboration, Bridget C. Andersen, Kevin Bandura, Mohit Bhardwaj, P. J. Boyle, Charanjot Brar, Tomas Cassanelli, S. Chatterjee, Pragya Chawla, Amanda M. Cook, Alice P. Curtin, Matt Dobbs, Fengqiu Adam Dong, Jakob T. Faber, Mateus Fandino, Emmanuel Fonseca, B. M. Gaensler, Utkarsh Giri, Antonio Herrera-Martin, Alex S. Hill, Adaeze Ibik, Alexander Josephy, Jane F. Kaczmarek, Zarif Kader, Victoria Kaspi, T. L. Landecker, Adam E. Lanman, Mattias Lazda, Calvin Leung, Hsiu-Hsien Lin, Kiyoshi W. Masui, Ryan Mckinven, Juan Mena-Parra, Bradley W. Meyers, D. Michilli, Cherry Ng, Ayush Pandhi, Aaron B. Pearlman, Ue-Li Pen, Emily Petroff, Ziggy Pleunis, Masoud Rafiei-Ravandi, Mubdi Rahman, Scott M. Ransom, Andre Renard, Ketan R. Sand, Pranav Sanghavi, Paul Scholz, Vishwangi Shah, Kaitlyn Shin, Seth Siegel, Kendrick Smith, Ingrid Stairs, Jianing Su, Shriharsh P. Tendulkar, Keith Vanderlinde, Haochen Wang, Dallas Wulf, and Andrew Zwaniga. Chime/frb discovery of 25 repeating fast radio burst sources. *The Astrophysical Journal*, 947(2):83, April 2023.
- [7] J. M. Cordes and T. J. W. Lazio. Ne2001.i. a new model for the galactic distribution of free electrons and its fluctuations, 2003.
- [8] V. Gajjar, A. P. V. Siemion, D. C. Price, C. J. Law, D. Michilli, J. W. T. Hessels, S. Chatterjee, A. M. Archibald, G. C. Bower, C. Brinkman, S. Burke-Spolaor, J. M. Cordes, S. Croft, J. Emilio Enriquez, G. Foster, N. Gizani, G. Hellbourg, H. Isaacson, V. M. Kaspi, T. J. W. Lazio, M. Lebofsky, R. S. Lynch, D. MacMahon, M. A. McLaughlin, S. M. Ransom, P. Scholz, A. Seymour, L. G. Spitler, S. P. Tendulkar, D. Werthimer, and Y. G. Zhang. Highest frequency detection of frb 121102 at 4–8 ghz using the breakthrough listen digital backend at the green bank telescope. *The Astrophysical Journal*, 863(1):2, August 2018.
- [9] Khee-Gan Lee, Metin Ata, Ilya S. Khrykin, Yuxin Huang, J. Xavier Prochaska, Jeff Cooke, Jielai Zhang, and Adam Batten. Constraining the cosmic baryon distribution with fast radio burst foreground mapping. *The Astrophysical Journal*, 928(1):9, March 2022.
- [10] D. R. Lorimer, M. Bailes, M. A. McLaughlin, D. J. Narkevic, and F. Crawford. A bright millisecond radio burst of extragalactic origin. *Science*, 318(5851):777–780, November 2007.

- [11] J.-P. Macquart, J. X. Prochaska, M. McQuinn, K. W. Bannister, S. Bhandari, C. K. Day, A. T. Deller, R. D. Ekers, C. W. James, L. Marnoch, S. Osłowski, C. Phillips, S. D. Ryder, D. R. Scott, R. M. Shannon, and N. Tejos. A census of baryons in the universe from localized fast radio bursts. *Nature*, 581(7809):391–395, May 2020.
- [12] Y P Men, R Luo, M Z Chen, L F Hao, K J Lee, J Li, Z X Li, Z Y Liu, X Pei, Z G Wen, J J Wu, Y H Xu, R X Xu, J P Yuan, and C F Zhang. Piggyback search for fast radio bursts using nanshan 26m and kunming 40m radio telescopes – i. observing and data analysis systems, discovery of a mysterious peryton. *Monthly Notices of the Royal Astronomical Society*, 488(3):3957–3971, July 2019.
- [13] Yunpeng Men and Ewan Barr. Transientx: A high performance single pulse search package, 2024.
- [14] Hamsa Padmanabhan and Abraham Loeb. A new limit on intergalactic magnetic fields on subkiloparsec scales from fast radio bursts. *The Astrophysical Journal Letters*, 946(1):L18, March 2023.
- [15] E. Petroff, J. W. T. Hessels, and D. R. Lorimer. Fast radio bursts. *The Astronomy and Astrophysics Review*, 27(1), May 2019.
- [16] E. Petroff, J. W. T. Hessels, and D. R. Lorimer. Fast radio bursts at the dawn of the 2020s. *The Astronomy and Astrophysics Review*, 30(1), March 2022.
- [17] Z. Pleunis, D. Michilli, C. G. Bassa, J. W. T. Hessels, A. Naidu, B. C. Andersen, P. Chawla, E. Fonseca, A. Gopinath, V. M. Kaspi, V. I. Kondratiev, D. Z. Li, M. Bhardwaj, P. J. Boyle, C. Brar, T. Cassanelli, Y. Gupta, A. Josephy, R. Karuppusamy, A. Keimpema, F. Kirsten, C. Leung, B. Marcote, K. W. Masui, R. Mckinven, B. W. Meyers, C. Ng, K. Nimmo, Z. Paragi, M. Rahman, P. Scholz, K. Shin, K. M. Smith, I. H. Stairs, and S. P. Tendulkar. Lofar detection of 110–188 mhz emission and frequency-dependent activity from frb 20180916b. *The Astrophysical Journal Letters*, 911(1):L3, April 2021.
- [18] M. P. Snelders, K. Nimmo, J. W. T. Hessels, Z. Bensellam, L. P. Zwaan, P. Chawla, O. S. Ould-Boukattine, F. Kirsten, J. T. Faber, and V. Gajjar. Detection of ultra-fast radio bursts from frb 20121102a. *Nature Astronomy*, 7(12):1486–1496, October 2023.
- [19] J. M. Yao, R. N. Manchester, and N. Wang. A new electron-density model for estimation of pulsar and frb distances. *The Astrophysical Journal*, 835(1):29, January 2017.
- [20] Ze-Wei Zhao, Ling-Feng Wang, Ji-Guo Zhang, Jing-Fei Zhang, and Xin Zhang. Probing the interaction between dark energy and dark matter with future fast radio burst observations. *Journal of Cosmology and Astroparticle Physics*, 2023(04):022, April 2023.